

std::basic_const_iterator should follow its underlying type's convertibility

Document #: P2836R1
Date: 2023-07-08
Project: Programming Language C++
Audience: Ranges Study Group (SG9)
Library Evolution Working Group (LEWG)
Library Working Group (LWG)
Reply-to: Christopher Di Bella
<cjdb@google.com>

Contents

- [1 Introduction](#)
- [2 Acknowledgements](#)
 - [2.1 R1](#)
 - [2.2 R0](#)
- [3 Changes between R0 and R1](#)
 - [3.1 Recap](#)
 - [3.2 Change in design \(and title\)](#)
 - [3.3 Which standard is this targeting?](#)
- [4 Problem](#)
- [5 Proposed resolution](#)
- [6 Proposed wording](#)
 - [6.1 Changes to \[version.syn\]](#)
 - [6.2 Changes to \[const.iterators.iterator\]](#)
 - [6.3 Changes to \[const.iterators.ops\]](#)
- [7 Implementation experience](#)
- [8 References](#)

§ 1 Introduction

Many iterators can be implicitly converted to their corresponding `const_iterator`—such as the standard containers’—but `basic_const_iterator` doesn’t permit that (yet).

§ 2 Acknowledgements

§ 2.1 R1

Thank you to Barry Revzin and Tomasz Kamiński for their patience in explaining why the direction outlined in R0 wasn’t feasible. Also thanks for helping to tease out the parts that could be fixed via a [defect report](#), rather than a full-on proposal.

Double thanks to Tomasz, who has agreed to represent this paper in my absence.

§ 2.2 R0

Thank you to Richard Smith, Janet Cobb, Nicole Mazzuca, and David Blaikie for providing feedback on the text and design of the proposed resolution.

Thank you to Nicole and Stephan T. Lavavej (STL) for reviewing the CL that implements the proposed fix in my Microsoft/STL fork.

§ 3 Changes between R0 and R1

§ 3.1 Recap

[P2836R0] (previous revision) wanted to introduce a trait-like type to identify an iterator’s corresponding `const_iterator`, and use that to limit the number of redundant function template instantiations and debug info that a program would generate from using `basic_const_iterator`.

§ 3.2 Change in design (and title)

Barry and Tomasz both pointed out over emails (some of which are on the reflector) that this trait wouldn’t address the following situation:

```
namespace stdv = std::views;

auto v = std::vector<int>();
auto t = v | stdv::take_while([](int const x) { return x < 100; });

v.begin() == t.end(); // okay
v.cbegin() == t.end(); // error, see Tomasz's example from C++20:
                        https://godbolt.org/z/Txa5cGYYY
```

Tomasz explained that having `std::const_iterator<std::vector<int>::iterator>` being `std::vector<int>::const_iterator` would cause problems for `t | stdv::as_const` in a similar way.

This would be a crippling blow to the utility of ranges, and I've received input from someone more experienced than myself in the area of optimising debug info and program sizes, that while the added redundant function template instantiations and extra debug info aren't great, they're not likely to be antagonistic here.

With this in mind, R1 (this revision) proposes the alternative design considered.

§ 3.3 Which standard is this targeting?

P2836R0 required a retroactive fix to C++23. This iteration has no such requirement, so we're now aiming for C++26, with encouragement to implementers to backport this to C++23.

§ 4 Problem

Consider

```
auto f = [](std::vector<int>::const_iterator i) {};  
  
auto v = std::vector<int>();  
{  
    auto i1 = std::ranges::cbegin(v); // returns vector<T>::const_iterator  
    f(i1); // okay  
}  
  
auto t = v | std::views::take_while([](int const x) { return x < 100; });  
{  
    auto i2 = std::ranges::cbegin(t); // returns  
    basic_const_iterator<vector<T>::iterator>  
    f(i2); // error in C++23  
}
```

The first call to `f` is allowed because `vector<int>::iterator` is implicitly convertible to `vector<int>::const_iterator`. `basic_const_iterator` doesn't have any conversion operators, and so we can't convert `basic_const_iterator<vector<int>::iterator>` to `vector<int>::const_iterator`, despite them being the same logical type.

Note that this works in C++20, but that was because `ranges::cbegin(t)` returned `vector<int>::iterator` instead ([P2278R4] was a C++23 addition).

§ 5 Proposed resolution

This paper proposes to make it possible for `basic_const_iterator<I>` to be implicitly convertible to any constant iterator that `I` can be implicitly and explicitly convertible to. This solves the above problem in a non-intrusive manner to users, while also keeping the const model.

The paper also proposes that this be considered library DR, so that C++20 code won't be broken during migration to C++23.

§ 6 Proposed wording

§ 6.1 Changes to [version.syn]

```
- #define __cpp_lib_ranges_as_const          202207L // also in
  <ranges>
+ #define __cpp_lib_ranges_as_const          ??????L // also in
  <ranges>
```

§ 6.2 Changes to [const.iterators.iterator]

```
// ...
template<sentinel_for<Iterator> S>
constexpr bool operator==(const S& s) const;

+ template<not-a-const-iterator CI>
+   requires constant_iterator<CI> && convertible_to<Iterator const&,
+   CI>
+   constexpr operator CI() const&;
+ template<not-a-const-iterator CI>
+   requires constant_iterator<CI> && convertible_to<Iterator, CI>
+   constexpr operator CI() &&;

constexpr bool operator<(const basic_const_iterator& y) const
  requires random_access_iterator<Iterator>;
// ...
```

§ 6.3 Changes to [const.iterators.ops]

```
template<sentinel_for<Iterator> S>
constexpr bool operator==(const S& s) const;
```

Effects: Equivalent to: return current_ == s;

```
template<not-a-const-iterator CI>
  requires constant_iterator<CI> && convertible_to<Iterator const&, CI>
constexpr operator CI() const&;
```

Returns: current_.

```
template<not-a-const-iterator CI>
  requires constant_iterator<CI> && convertible_to<Iterator, CI>
constexpr operator CI() &&;
```

Returns: std::move(current_).

§ 7 Implementation experience

The above wording has been implemented and tested using [a Microsoft/STL fork](#).

§ 8 References

[P2278R4] Barry Revzin. 2022-06-17. cbegin should always return a constant iterator.

<https://wg21.link/p2278r4>

[P2836R0] Christopher Di Bella. 2023-03-21. std::const_iterator often produces an unexpected type.

<https://wg21.link/p2836r0>